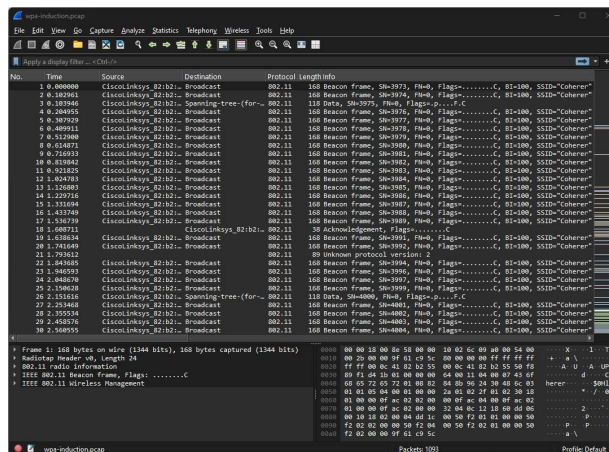


Lab 6

Stephanie Salgado

Opening the packet file on Wireshark



I open the file with no errors noted.

Using display filters

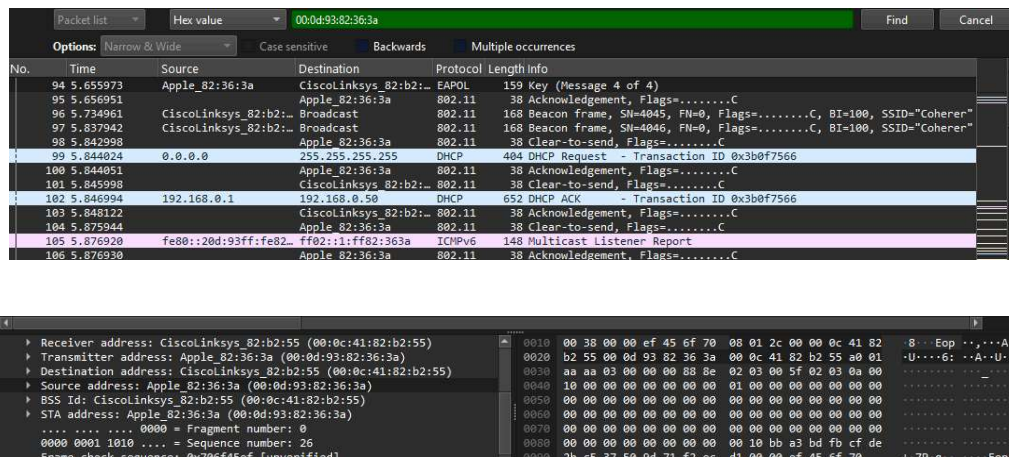
Filtering by IP address



No.	Time	Source	Destination	Protocol	Length	Info
102	5.846994	192.168.0.1	192.168.0.50	DHCP	652	DHCP ACK - Transaction ID 0x3b0f7566
265	8.439534	192.168.0.50	192.168.0.1	UDP	108	50068 → 192 Len=4
268	8.440534	192.168.0.1	192.168.0.50	ICMP	136	Destination unreachable (Port unreachable)
305	8.937475	192.168.0.50	192.168.0.1	UDP	108	50070 → 192 Len=4
308	8.938464	192.168.0.1	192.168.0.50	ICMP	136	Destination unreachable (Port unreachable)
330	9.944239	192.168.0.50	192.168.0.1	UDP	108	50071 → 192 Len=4
333	9.946244	192.168.0.1	192.168.0.50	ICMP	136	Destination unreachable (Port unreachable)
341	10.442158	192.168.0.50	192.168.0.1	UDP	108	50072 → 192 Len=4
344	10.443158	192.168.0.1	192.168.0.50	ICMP	136	Destination unreachable (Port unreachable)
375	10.943080	192.168.0.50	192.168.0.1	UDP	108	50074 → 192 Len=4
378	10.944072	192.168.0.1	192.168.0.50	ICMP	136	Destination unreachable (Port unreachable)
390	11.450990	192.168.0.50	192.168.0.1	UDP	108	50075 → 192 Len=4
393	11.451981	192.168.0.1	192.168.0.50	ICMP	136	Destination unreachable (Port unreachable)
406	12.446856	192.168.0.50	192.168.0.1	UDP	108	50076 → 192 Len=4
408	12.447825	192.168.0.1	192.168.0.50	ICMP	136	Destination unreachable (Port unreachable)
415	12.947797	192.168.0.50	192.168.0.1	UDP	108	50077 → 192 Len=4
417	12.948755	192.168.0.1	192.168.0.50	ICMP	136	Destination unreachable (Port unreachable)
419	12.954731	192.168.0.50	192.168.0.1	UDP	108	50078 → 192 Len=4
421	12.955740	192.168.0.1	192.168.0.50	ICMP	136	Destination unreachable (Port unreachable)
422	12.956730	192.168.0.1	192.168.0.50	ICMP	136	Destination unreachable (Port unreachable)
523	14.461493	192.168.0.50	192.168.0.1	UDP	108	50081 → 192 Len=4
526	14.463482	192.168.0.1	192.168.0.50	ICMP	136	Destination unreachable (Port unreachable)

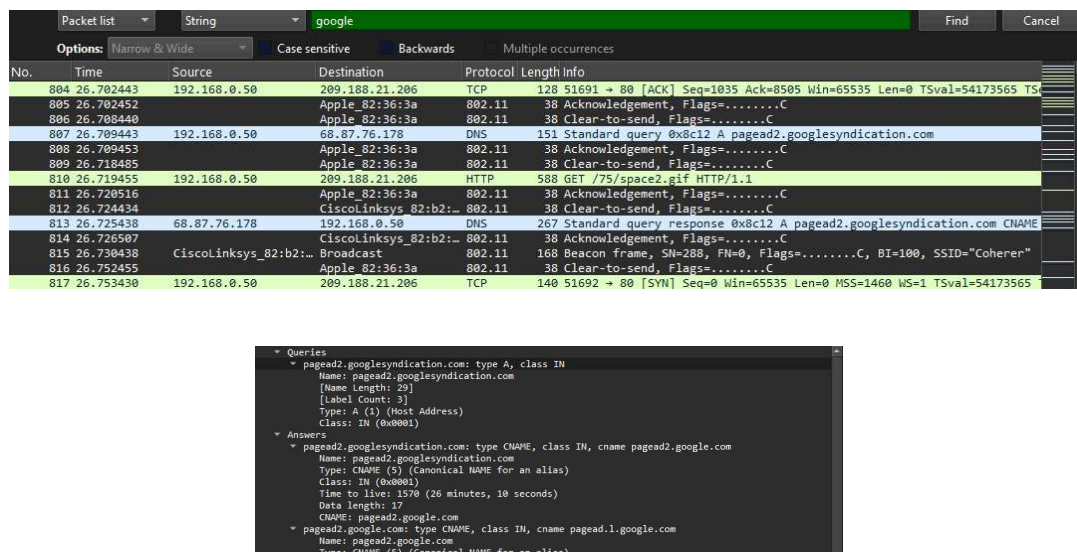
I use "ip.addr == 192.168.0.1" in the display filter box to filter the packets by the IP address "192.168.0.1", quickly revealing packets associated with it.

Filtering by HEX value



Navigating to "Edit" > "Find Packet" allows for more complicated filtering. Here is where I filter by the HEX value "00:0d:93:82:36:3a".

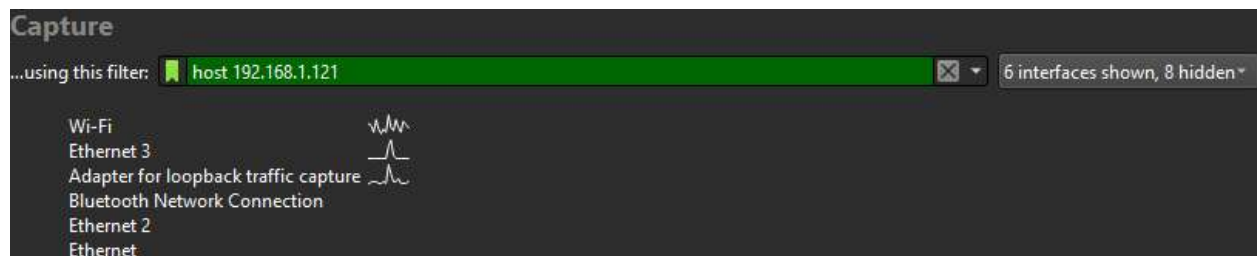
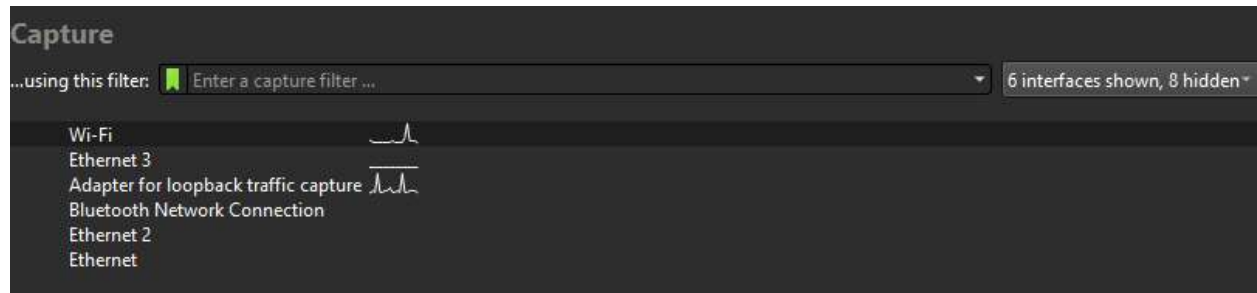
Filtering by String



Similarly, Wireshark allows users to filter by string. In this case "google". This is useful when trying to find traffic specifically pertaining to google. Take packet "813". Closer inspection reveals a query to "pagead2.googleadsyndication.com", and reveals which answers it received.

Using capture filters

Capturing traffic on my home network (Wi-Fi)



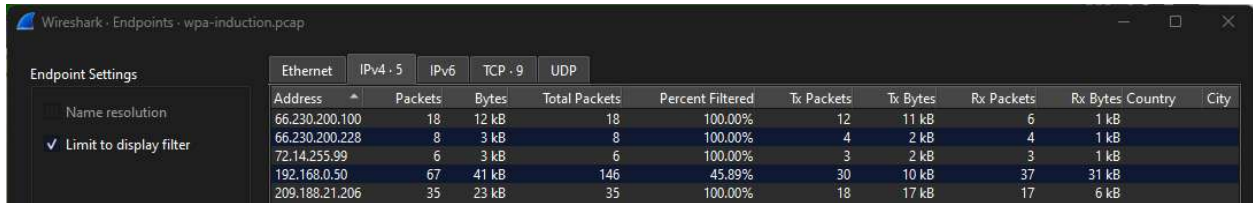
There are 2 ways of doing this. Wireshark allows you to capture from interfaces it has found, but it also allows you to add your own “capture filter”. In this case, I can use the default “Wi-Fi” or use “host 192.168.1.121”.

Capturing from Wi-Fi (host 192.168.1.121)

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.121	162.247.243.29	TCP	55	52091 → 443 [ACK] Seq=1 Ack=1 Win=517 Len=1
2	0.018599	162.247.243.29	192.168.1.121	TCP	66	443 → 52091 [ACK] Seq=1 Ack=2 Win=332 Len=0 SLE=1 SRE=2
3	0.641311	192.168.1.121	18.209.201.158	TLSv1.2	125	Application Data
4	0.691848	18.209.201.158	192.168.1.121	TCP	54	443 → 49852 [ACK] Seq=1 Ack=72 Win=8 Len=0
5	0.691848	18.209.201.158	192.168.1.121	TLSv1.2	145	Application Data
6	0.746199	192.168.1.121	18.209.201.158	TCP	54	49852 → 443 [ACK] Seq=72 Ack=92 Win=507 Len=0
7	1.047817	AskeyCompute_bc:f6:...	Intel_dc:ab:68	ARP	42	Who has 192.168.1.121? Tell 192.168.1.1
8	1.047831	Intel_dc:ab:68	AskeyCompute_bc:f6:...	ARP	42	192.168.1.121 is at e8:f4:08:dc:ab:68
9	4.180845	20.241.44.114	192.168.1.121	TCP	54	443 → 52103 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
10	5.416229	18.209.201.158	192.168.1.121	TCP	54	[TCP Keep-Alive] 443 → 49852 [ACK] Seq=91 Ack=72 Win=8 Len=0
11	5.416242	192.168.1.121	18.209.201.158	TCP	54	[TCP Keep-Alive ACK] 49852 → 443 [ACK] Seq=72 Ack=92 Win=507 Len=0
12	10.641411	192.168.1.121	18.209.201.158	TLSv1.2	125	Application Data
13	10.735891	18.209.201.158	192.168.1.121	TCP	54	443 → 49852 [ACK] Seq=92 Ack=143 Win=8 Len=0
14	10.812529	54.242.246.200	192.168.1.121	TCP	54	443 → 52072 [FIN, ACK] Seq=1 Ack=1 Win=140 Len=0
15	10.812529	54.242.246.200	192.168.1.121	TCP	100	[TCP Out-Of-Order] 443 → 52072 [PSH, ACK] Seq=4294967220 Ack=1 Win=140 Len=0
16	10.812529	54.242.246.200	192.168.1.121	TCP	85	[TCP Out-Of-Order] 443 → 52072 [PSH, ACK] Seq=4294967266 Ack=1 Win=140 Len=0
17	10.812563	192.168.1.121	54.242.246.200	TCP	54	52072 → 443 [ACK] Seq=1 Ack=4294967220 Win=513 Len=0
18	10.812609	192.168.1.121	54.242.246.200	TCP	54	52072 → 443 [ACK] Seq=1 Ack=2 Win=512 Len=0
19	10.812744	192.168.1.121	54.242.246.200	TCP	54	52072 → 443 [FIN, ACK] Seq=1 Ack=2 Win=512 Len=0
20	10.868615	54.242.246.200	192.168.1.121	TCP	54	443 → 52072 [ACK] Seq=2 Ack=2 Win=140 Len=0

It begins capturing all traffic.

Analyzing endpoints (“wpa-induction.pcap”)



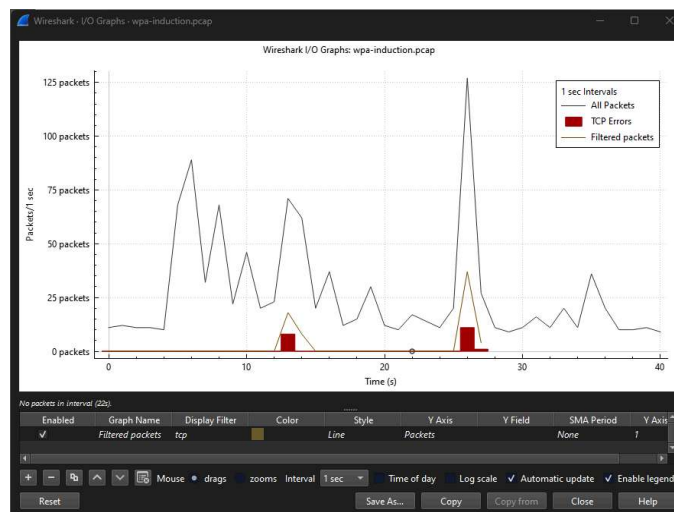
Address	Packets	Bytes	Total Packets	Percent Filtered	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes	Country	City
66.230.200.100	18	12 kB	18	100.00%	12	11 kB	6	1 kB		
66.230.200.228	8	3 kB	8	100.00%	4	2 kB	4	1 kB		
72.14.255.99	6	3 kB	6	100.00%	3	2 kB	3	1 kB		
192.168.0.50	67	41 kB	146	45.89%	30	10 kB	37	31 kB		
209.188.21.206	35	23 kB	35	100.00%	18	17 kB	17	6 kB		

I select a TCP packet (444), then go to “Statistics” > “Endpoints” > “IPv4”. From here, I noticed “192.168.0.50” received the most traffic at 67 packets. I switched to the “IPv6” tab but found nothing.

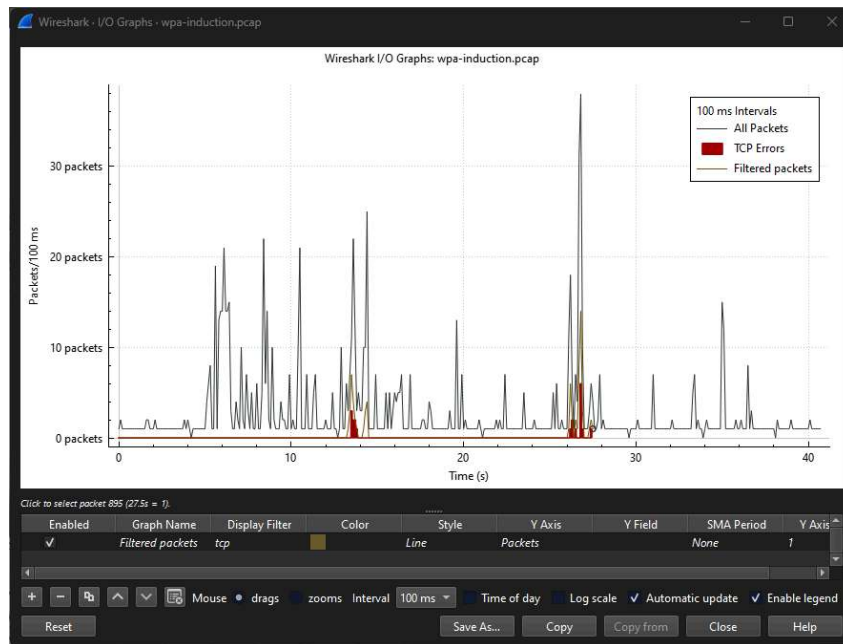
Analyzing graphs for traffic

The graphs can be found under “Statistics”.

IO graphs

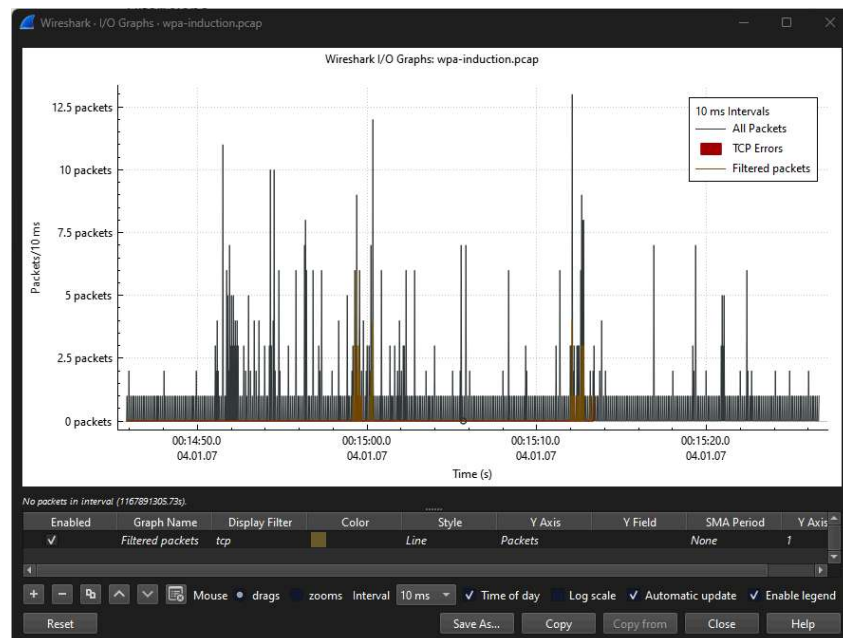


Filter: None



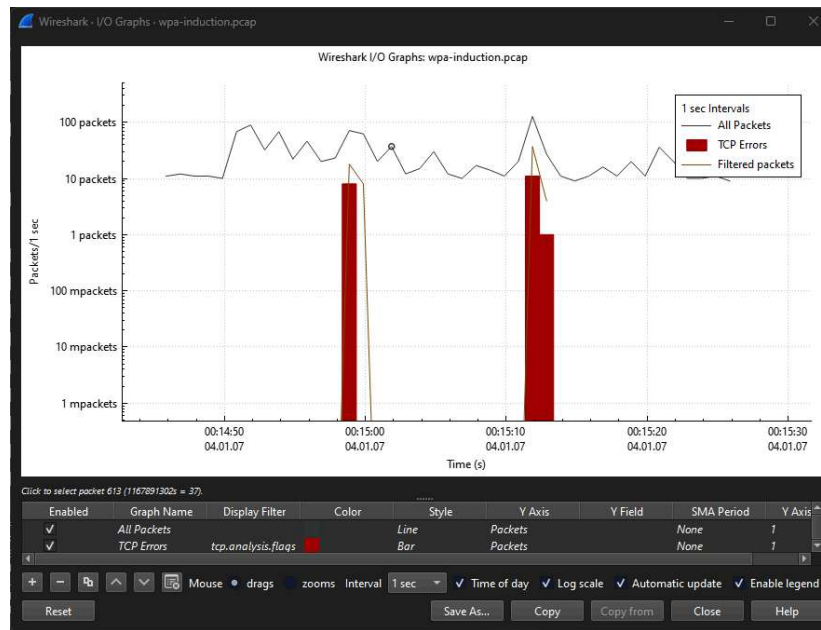
Filter: 100ms Interval

Notice at 100ms, the graph seems more detailed.



Filter: 10s Interval, Time of day

With this filter, the graph has become increasingly difficult to read.



Filter: 1s Interval, Time of day, Log scale

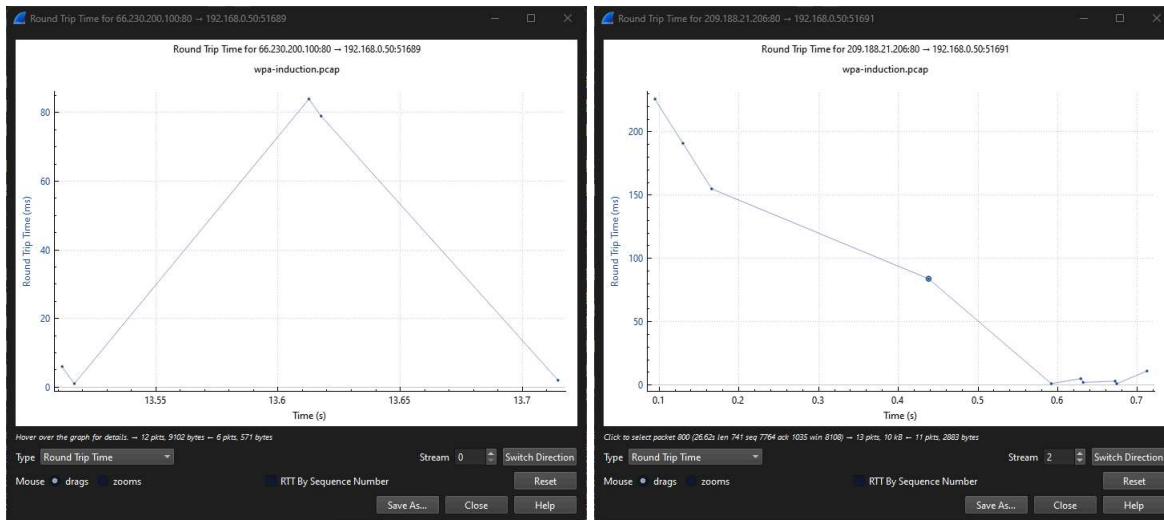
With this final filter, the graph becomes more legible.

Flow graphs

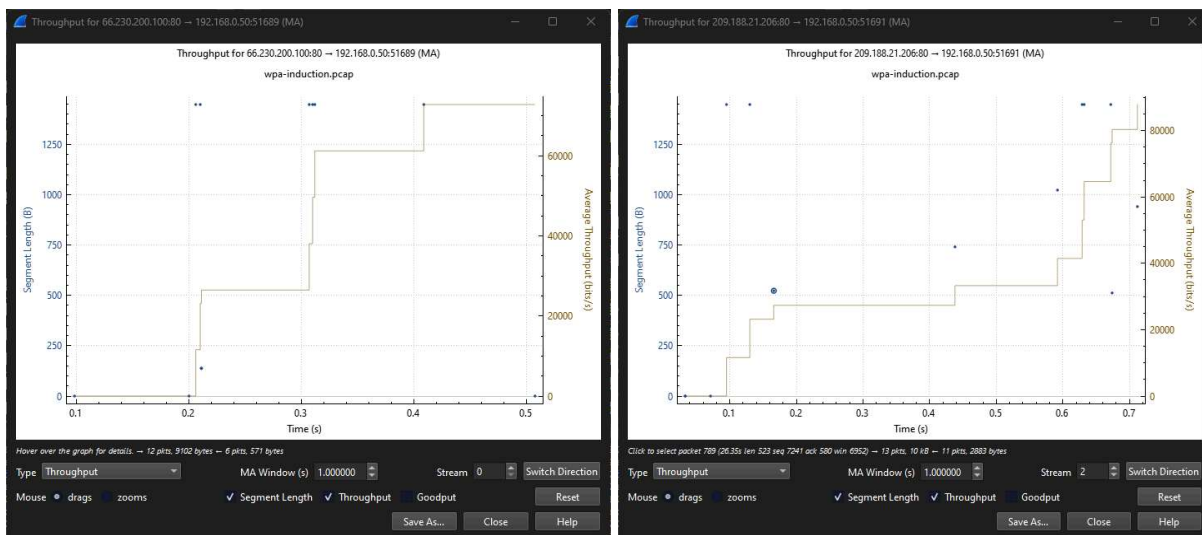


Filters can also be applied, but viewing it as a whole may be more useful.

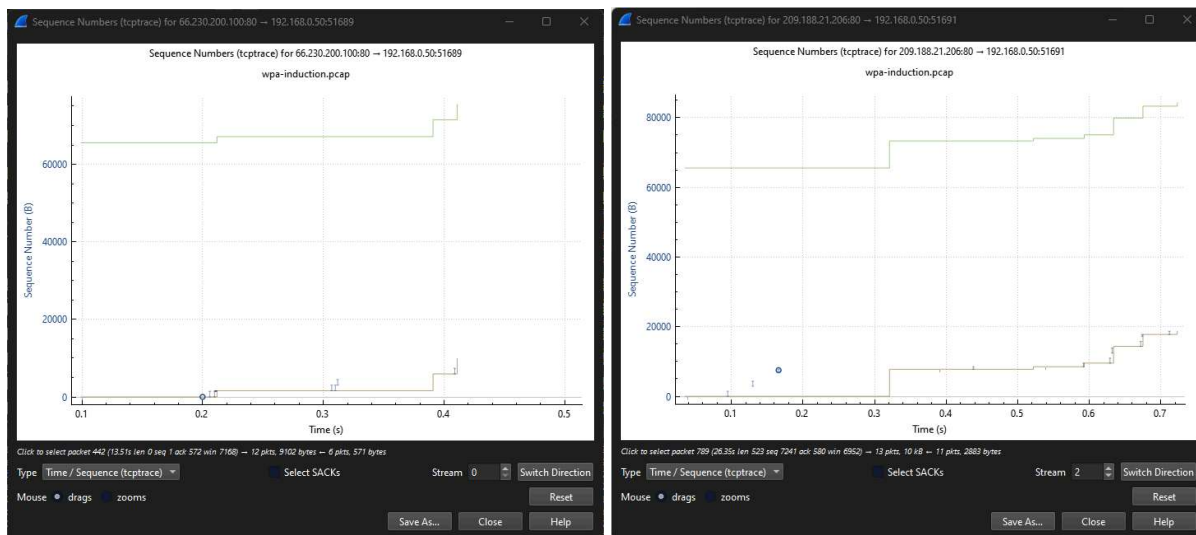
TCP stream graphs (RTT, Throughput, Time sequence)



Round-trip time (RTT) is the time it takes to receive an acknowledgment (ACK) for a sent packet in TCP communication. The x-axis of the graph shows the TCP sequence number, while the y-axis shows the RTT in seconds, with the plotted points helping to identify latency issues.



Throughput graphs show the direction of network traffic, focusing on unidirectional flow, unlike IO graphs that display traffic in both directions. The data on the graph varies based on the selected packet.



The time sequence graph (tcptrace) displays the flow of TCP traffic over time, focusing on unidirectional data. It helps identify segments in transit, ACKs received, and the client's buffer capacity. There's another type of time sequence graph (Stevens), which shows similar info in another format.

TCP stream graphs observations

I used 2 packets to note any differences. Packet 444 had a large peak on its RTT graph, while packet 804 had a downwards slope all the way through. Both their throughput and time sequence (tcptrace) graphs were different as well.

Summary

In this lab, I used the same packet file from Lab 2. I also captured some of my home network traffic. I learned that IO graphs can help identify peaks and valleys in traffic, which can be very important in monitoring. Flow graphs are column-type visualizations that help troubleshoot issues like dropped connections, lost frames, and retransmission traffic among other things. TCP stream graphs provide even more useful information. They help identify issues like round-trip time and retransmissions. I enjoyed this lab overall.